

Implementing Forward and Backward Propagation

CS4152 — Deep Learning and Neural Networks

Syed Hamed Raza

University of Management and Technology (UMT), Lahore

November 22, 2025

Outline

- 1 Forward Propagation
- 2 Backward Propagation
- 3 Vectorized Implementation
- 4 End-to-End Flow
- 5 Conclusion

Forward Function for Layer l

- Inputs: $A^{[l-1]}, W^{[l]}, b^{[l]}$
- Compute:

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

- Cache stores: $Z^{[l]}$ (and optionally $W^{[l]}, b^{[l]}$)
- Supports vectorized computation for m examples.

Forward Pass Through the Network

- Initialize: $A^{[0]} = X$
- Sequential propagation:

$$A^{[1]} \rightarrow A^{[2]} \rightarrow \dots \rightarrow A^{[L]}$$

- Final output: $\hat{y} = A^{[L]}$
- Cache from each layer stored for backprop.

Backward Function for Layer l

- Inputs: $dA^{[l]}$, cache $(Z^{[l]}, W^{[l]}, b^{[l]})$
- Compute:

$$dZ^{[l]} = dA^{[l]} \odot g'^{[l]}(Z^{[l]})$$

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T$$

$$db^{[l]} = \frac{1}{m} \sum dZ^{[l]}$$

$$dA^{[l-1]} = (W^{[l]})^T dZ^{[l]}$$

Backward Pass Through the Network

- Start with:

$$dA^{[L]} = \frac{Y}{A^{[L]}} - \frac{1 - Y}{1 - A^{[L]}}$$

- Backward sequence:

$$dA^{[L]} \rightarrow dA^{[L-1]} \rightarrow \dots \rightarrow dA^{[1]}$$

- Each layer yields:

$$dW^{[l]}, db^{[l]}, dA^{[l-1]}$$

- $dA^{[0]}$ is discarded.

Vectorized Backward Formulas

- $dZ^{[l]} = dA^{[l]} \odot g'^{[l]}(Z^{[l]})$
- $dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T$
- $db^{[l]} = \frac{1}{m} \sum dZ^{[l]}$, axis = 1, keepdims=True
- $dA^{[l-1]} = (W^{[l]})^T dZ^{[l]}$
- Fully compatible with batch processing.

Combined Forward–Backward Pipeline

- Forward pass $\rightarrow$ output $\hat{y}$
- Compute loss and $dA^{[L]}$
- Backward pass:

$$(dW^{[1]}, db^{[1]}), \dots, (dW^{[L]}, db^{[L]})$$

- Update parameters using gradient descent.
- Entire training loop is built from these reusable blocks.

Key Takeaways

- Forward and backward functions form modular building blocks.
- Cache mechanism connects forward and backward passes.
- Vectorization ensures efficient training on large datasets.
- Implementation becomes intuitive after coding exercises.